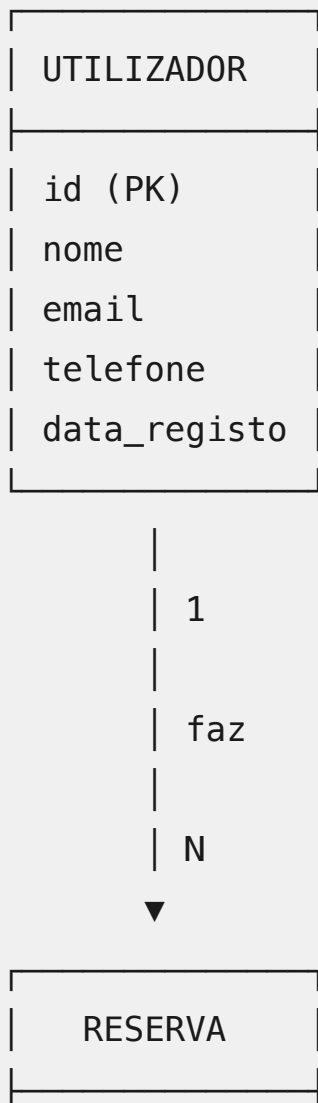


# SOLUÇÕES - EXAME MODELO DE BASES DE DADOS

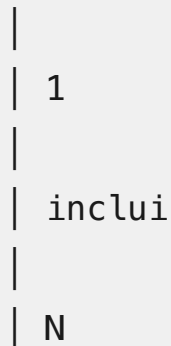
## QUESTÃO 1 (3 valores)

### a) Diagrama Entidade-Associação (2 valores)

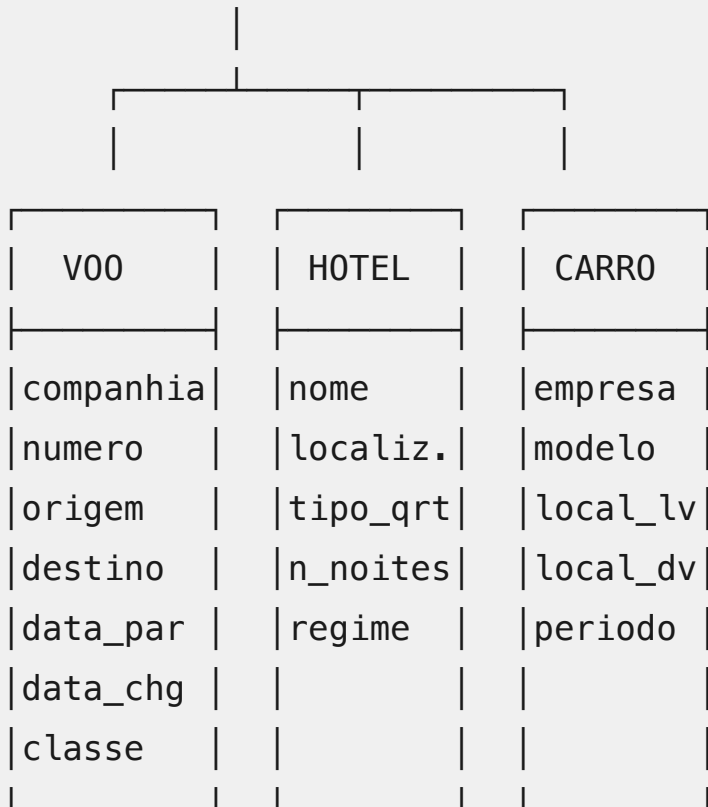
Solução:



|              |
|--------------|
| id (PK)      |
| data_reserva |
| preco_total  |
| estado       |



|                    |
|--------------------|
| ITEM_RESERVA       |
| (Especialização ou |
| Generalização)     |



**Nota:** Existem várias soluções válidas. O importante é:

- Separar as entidades principais (Utilizador, Reserva)
- Modelar corretamente que uma reserva pode ter vários itens
- Usar especialização/generalização OU relações separadas para Voo/Hotel/Carro
- Indicar cardinalidades corretas (1:N entre Utilizador-Reserva, 1:N entre Reserva-Itens)

## **b) Justificação de decisão (1 valor)**

### **Exemplos de respostas válidas:**

#### **1. Usando Generalização/Especialização:**

- "Escolhi usar generalização/especialização (IS-A) para modelar Voo, Hotel e Carro como subtipos de ItemReserva porque partilham alguns atributos comuns (como preço, data) mas cada um tem atributos específicos muito diferentes. Esta abordagem evita ter muitos atributos NULL numa única tabela."

#### **2. Usando Entidades Separadas:**

- "Optei por criar três entidades separadas (ItemVoo, ItemHotel, ItemCarro) ligadas a Reserva porque cada tipo de item tem atributos completamente diferentes e não há benefício em forçá-los numa hierarquia. Isto também facilita queries específicas para cada tipo de item."

---

## **QUESTÃO 2 (3 valores)**

### **a) Interpretação da query (1 valor)**

**Solução:**

A query retorna os **top 100 utilizadores que gastaram mais de 1000€ em reservas a partir de 1 de janeiro de 2024**, ordenados por gasto total (do maior para o menor).

### Explicação das cláusulas:

- **LEFT JOIN:** Junta todos os utilizadores com as suas reservas (mantém utilizadores mesmo sem reservas)
- **WHERE r.data\_reserva >= '2024-01-01':** Filtra apenas reservas feitas em 2024 ou depois
- **GROUP BY:** Agrupa os resultados por utilizador (id, nome, email)
- **HAVING SUM(r.preco\_total) > 1000:** Filtra apenas grupos onde o gasto total é superior a 1000€
- **ORDER BY gasto\_total DESC:** Ordena do maior gasto para o menor
- **LIMIT 100:** Retorna apenas os primeiros 100 resultados

### b) Problema na query (1 valor)

#### Potenciais causas:

- **LIMIT 100** limita os resultados aos 100 clientes que gastaram mais. O utilizador pode não ser um dos 100 primeiros, e por isso não aparece na lista.
- **WHERE r.data\_reserva >= '2024-01-01'** omite reservas feitas antes de 1 de janeiro de 2024, mesmo que o pagamento da reserva tenha sido feito depois dessa data.

#### Correção:

- Para a primeira causa: Remover o LIMIT
- Para a segunda causa: Acrescentar uma entidade Pagamento e atualizar a interrogação para verificar a data do pagamento em vez da data da reserva

## c) Índices para melhorar performance (1 valor)

### Solução:

**Índice 1:** `CREATE INDEX idx_reserva_data_user ON Reserva(data_reserva, id_utilizador);`

- Melhora a filtragem por data e facilita o JOIN com Utilizador
- A ordem (data primeiro) permite filtrar eficientemente as reservas de 2024

**Índice 2:** `CREATE INDEX idx_reserva_preco ON Reserva(preco_total);`

- Acelera o cálculo de SUM(preco\_total) na agregação
- Útil para o HAVING que filtra por soma de preços

### Alternativas válida:

- `CREATE INDEX idx_reserva_user ON Reserva(id_utilizador);` - melhora o JOIN
  - Índice composto incluindo `preco_total`: `CREATE INDEX idx_reserva_complete ON Reserva(data_reserva, id_utilizador, preco_total);`
- 

## QUESTÃO 3 (3 valores)

### a) Três problemas específicos (1.5 valores)

#### Solução:

#### 1. Redundância de dados:

- Os dados do utilizador (nome, email, telefone) estão **duplicados em cada reserva**

- Impacto: Se um utilizador tem 100 reservas, os seus dados pessoais estão armazenados 100 vezes, desperdiçando espaço

## 2. Anomalias de atualização:

- Se o utilizador mudar de email ou telefone, é necessário atualizar **todas as suas reservas**
- Impacto: Risco de inconsistências se algumas reservas forem atualizadas e outras não

## 3. Anomalias de inserção/eliminação:

- **Inserção 1:** Não é possível criar uma reserva com mais do que um voo, hotel ou carro
- **Inserção 2:** Não é possível criar uma reserva sem preencher todos os campos de voo, hotel e carro, mesmo que a reserva seja apenas de hotel (resultando em muitos valores NULL)
- **Eliminação:** Se apagarmos todas as reservas de um utilizador, perdemos também a informação do utilizador
- Exemplo: Uma reserva apenas de hotel obriga a ter os campos voo\_companhia, voo\_numero, carro\_empresa, carro\_modelo como NULL, desperdiçando espaço

## b) Por que razão o novo esquema é melhor (1.5 valores)

### Solução:

O novo esquema é melhor pelas seguintes razões:

### 1. Elimina redundância de dados do utilizador:

- Os dados pessoais (nome, email, telefone) são armazenados apenas uma vez na tabela Utilizador
- Para milhões de utilizadores com dezenas de reservas cada, isto representa uma enorme poupança de espaço
- Facilita atualizações: mudar o email de um utilizador requer apenas 1 UPDATE em vez de N UPDATES (onde N é o número de reservas)

do utilizador)

## **2. Evita valores NULL desnecessários:**

- Cada reserva tem apenas os itens que realmente contém (voo, hotel, ou carro)
- Não há campos vazios/NULL para tipos de itens que não fazem parte da reserva
- Mais eficiente em termos de espaço de armazenamento

## **3. Melhora integridade dos dados:**

- A informação do utilizador não pode ficar inconsistente entre reservas
- Atualizações de dados pessoais são automáticas para todas as reservas (via chave estrangeira)
- Não há risco de ter emails diferentes para o mesmo utilizador em reservas diferentes

## **4. Flexibilidade:**

- Fácil adicionar novos tipos de itens (ex: seguro de viagem, transfers) sem alterar a estrutura existente
- Uma reserva pode ter múltiplos voos, múltiplos hotéis, etc.

---

# **QUESTÃO 4 (3 valores)**

## **a) Problema de concorrência e consequências (1 valor)**

### **Problema identificado:**

O problema é uma atualização perdida. Ambas as transações leem que existem 2 lugares disponíveis, ambas fazem reservas, e ambas atualizam para 1 lugar disponível. O resultado final é 1 lugar disponível,

mas na realidade foram vendidas 2 reservas, resultando em **overbooking** (3 lugares vendidos para um voo com apenas 2 lugares).

### Sequência do problema:

1. T1 e T2 leem simultaneamente: 2 lugares disponíveis
2. T1 e T2 confirmam ambas as reservas (pensando que há lugares)
3. T1 atualiza: lugares = 1
4. T2 atualiza: lugares = 1 (sobrepõe a atualização de T1)
5. Resultado: 2 reservas confirmadas mas apenas 1 lugar marcado como ocupado

### Consequências práticas:

1. **Problema operacional:** No dia do voo, 3 passageiros aparecem para 2 lugares
2. **Compensações financeiras:** A companhia tem de compensar o passageiro excedente
3. **Danos reputacionais:** Cliente insatisfeito, reviews negativas
4. **Custos legais:** Possíveis penalizações por overbooking

## b) Duas abordagens para resolver (1 valor)

### Abordagem 1: Locking Pessimista (SELECT FOR UPDATE)

```
BEGIN TRANSACTION;  
SELECT lugares_disponiveis  
FROM Voo  
WHERE id=747  
FOR UPDATE;  -- Bloqueia a linha  
  
-- Verificar se há lugares  
-- Fazer INSERT na Reserva  
-- Fazer UPDATE no Voo
```



COMMIT;

Como funciona: O `FOR UPDATE` adquire um lock exclusivo na linha, impedindo que T2 leia enquanto T1 está a processar. T2 fica em espera até T1 fazer COMMIT.

## Abordagem 2: Locking Otimista (com versioning/timestamp)

```
-- Adicionar coluna version à tabela Voo

UPDATE Voo
SET lugares_disponiveis = lugares_disponiveis - 1,
    version = version + 1
WHERE id = 747
    AND lugares_disponiveis > 0
    AND version = <versão_lida_anteriormente>;

-- Verificar se o UPDATE afetou 1 linha
-- Se afetou 0 linhas, houve conflito -> retry
```

Como funciona: Cada transação verifica se a versão ainda é a mesma antes de atualizar. Se outra transação já atualizou, o UPDATE falha e a aplicação pode fazer retry.

## c) Nível de isolamento READ COMMITTED (1 valor)

**Resposta:**

**Não, READ COMMITTED NÃO resolveria o problema.**

O nível READ COMMITTED apenas garante que:

- Não se leem dados não committed (evita dirty reads)
- Cada SELECT vê o estado committed mais recente

Mas NÃO garante que:

- Os dados permaneçam inalterados entre operações da mesma transação
- Não há race conditions em operações read-then-write

No cenário descrito, ambas as transações podem ler o valor committed (2 lugares), depois ambas fazem UPDATE, e a última sobrepõe a primeira.

### **Nível de isolamento adequado: SERIALIZABLE**

O nível SERIALIZABLE garante que as transações executam como se fossem em série (uma de cada vez), evitando completamente o problema de race conditions. T1 executaria completamente antes de T2 começar (ou vice-versa).

---

## **QUESTÃO 5 (3.5 valores)**

### **a) Escolha de tecnologia para cada requisito (2 valores)**

#### **R1: Histórico de pesquisas dos utilizadores**

- **Recomendação: MongoDB (NoSQL)**
- **Justificação:**
  - Estrutura variável: diferentes pesquisas têm diferentes parâmetros (voos têm origem/destino, hotéis têm localização/datas)
  - Volume elevado: potencialmente milhões de pesquisas por dia
  - Não requer transações ACID: histórico de pesquisas não precisa de consistência forte
  - Leituras frequentes para personalização: MongoDB é otimizado para read-heavy workloads

- Schema flexível permite adicionar novos tipos de pesquisas sem alterações

## **R2: Gestão de pagamentos e faturação**

- **Recomendação: PostgreSQL (Relacional)**
- **Justificação:**
  - **Requer transações ACID:** pagamentos exigem atomicidade e consistência forte
  - Integridade referencial crítica: ligações entre reservas, pagamentos, faturas devem ser garantidas
  - Dados estruturados e relacionados: tabelas de pagamentos, faturas, métodos de pagamento
  - Regulamentação: dados financeiros exigem auditing e compliance que SGBD relacionais garantem melhor
  - Queries complexas: relatórios financeiros, reconciliação, análise de receitas

## **R3: Catálogo de hotéis com reviews**

- **Recomendação: MongoDB (NoSQL)**
- **Justificação:**
  - Estrutura muito variável: hotéis diferentes têm atributos diferentes (alguns têm spa, piscina, restaurante; outros não)
  - Reviews aninhadas: faz sentido armazenar reviews como array dentro do documento do hotel
  - Schema flexível: fácil adicionar novos amenities sem alterar estrutura
  - Embedding: reviews podem estar embedded no documento do hotel para performance (1 query em vez de JOIN)
  - Escalabilidade horizontal: catálogo pode crescer muito, MongoDB escala facilmente

## **R4: Sistema de notificações em tempo real**

- **Recomendação: MongoDB (NoSQL)**

- **Justificação:**

- Escritas frequentes: notificações são criadas constantemente
- Schema flexível: diferentes tipos de notificações têm diferentes campos
- Alta disponibilidade: MongoDB garante disponibilidade para notificações críticas

## **b) Estrutura de documento JSON e vantagem (1.5 valores)**

### **Solução:**

```
{
  "_id": "U123456",
  "dados_pessoais": {
    "nome": "João Silva",
    "email": "joao.silva@example.com",
    "telefone": "+351912345678",
    "data_registo": "2023-05-15"
  },
  "preferencias": {
    "destinos_favoritos": ["Paris", "Londres",
"Barcelona"],
    "classe_preferida": "economica",
    "companhias_preferidas": ["TAP", "Ryanair"],
    "tipo_hotel": "4estrelas",
    "amenities": ["wifi", "pequeno_almoco", "ginasio"]
  },
  "ultimas_pesquisas": [
    {
      "tipo": "voo",
      "origem": "LIS",
      "destino": "PAR",
      "data": "2024-12-15",
```

```
    "timestamp": "2024-12-01T14:30:00Z"
  },
  {
    "tipo": "hotel",
    "cidade": "Paris",
    "check_in": "2024-12-20",
    "check_out": "2024-12-23",
    "timestamp": "2024-12-01T14:35:00Z"
  },
  {
    "tipo": "voo",
    "origem": "LIS",
    "destino": "LON",
    "data": "2025-01-10",
    "timestamp": "2024-12-05T10:00:00Z"
  }
],
"estatisticas": {
  "total_reservas": 15,
  "gasto_total": 4500.00,
  "ultima_reserva": "2024-11-20"
}
}
```

### Vantagem desta abordagem:

**1 query em vez de múltiplas queries/JOINS:** No modelo relacional, para obter toda esta informação seriam necessárias várias queries:

- SELECT do utilizador
- SELECT das preferências (tabela separada)
- SELECT das últimas pesquisas (tabela separada, ordenar por timestamp, LIMIT 3)
- SELECT de estatísticas (agregações em outras tabelas)

Com MongoDB, **1 única query** ( `db.users.findOne({_id: "U123456"})` ) retorna toda a informação, resultando em:

- Latência muito menor (1 round-trip à BD em vez de 4-5)
- Código mais simples na aplicação
- Melhor performance para casos de uso read-heavy

#### **Vantagens adicionais:**

- Schema flexível permite adicionar novos campos de preferências sem ter de migrar dados
  - Arrays permitem adicionar/remover destinos favoritos facilmente
  - Estrutura hierárquica (dados\_pessoais, preferencias) torna o código mais legível
- 

## **QUESTÃO 6 (2.5 valores)**

### **a) Comparação de estratégias (1.5 valores)**

#### **Estratégia A: Replicação Master-Slave**

##### **Vantagem principal:**

- Escalabilidade de leituras: múltiplos slaves podem servir as consultas de pesquisa (80% do workload), distribuindo a carga
- Simples de implementar e gerir
- Alta disponibilidade: se o master falhar, um slave pode ser promovido

##### **Desvantagem/Desafio:**

- Master continua a ser single point of failure e bottleneck para escritas (20% do workload)
- Todos os dados precisam existir em cada slave (não há particionamento)

- Replication lag: slaves podem estar ligeiramente desatualizados em relação ao master

### **Workload que beneficia:**

- **Read-heavy workloads** (como o da TravelEasy: 80% leituras)
  - Aplicações onde consistência eventual é aceitável para leituras
  - Sistemas com picos de leitura mas escritas relativamente constantes
- 

## **Estratégia B: Particionamento (Sharding) por região geográfica**

### **Vantagem principal:**

- Escalabilidade verdadeira: tanto leituras quanto escritas são distribuídas
- Latência reduzida: utilizadores europeus acedem a dados em servidores europeus (geograficamente próximos)
- Dados localizados: dados dos utilizadores ficam na sua região (compliance com GDPR)

### **Desvantagem/Desafio:**

- Muito mais complexo de implementar e gerir
- Queries cross-shard (ex: pesquisar voos globalmente) são mais lentas e complexas
- Rebalancing: redistribuir dados se o crescimento não for uniforme entre regiões

### **Workload que beneficia:**

- Aplicações com utilizadores geograficamente distribuídos
  - Workloads com muitas escritas (não apenas leituras)
  - Quando os dados podem ser naturalmente particionados (ex: por região, por cliente)
  - Aplicações que precisam de escalar tanto reads quanto writes
-

## Conclusão para a TravelEasy:

Para o workload atual (80% reads, 20% writes), **Estratégia A (Master-Slave)** é mais adequada como primeira solução porque:

- Resolve o problema imediato (escalar as leituras que são 80%)
- Muito mais simples de implementar
- Pode ser complementada depois com sharding se necessário

## b) Arquitetura para relatórios analíticos (1 valor)

### Solução:

#### Data Warehouse com pipeline ETL

#### Arquitetura recomendada:

1. **Base de dados de produção (OLTP):** PostgreSQL continua a servir a aplicação
2. **Data Warehouse (OLAP):** Sistema separado otimizado para queries analíticas
3. **Pipeline ETL:**
  - Extract: extrair dados da BD de produção durante períodos de baixo uso
  - Transform: agregar, limpar e transformar dados
  - Load: carregar no data warehouse
  - Executar todas as noites

### Justificação:

#### 1. Separation of concerns:

- Queries analíticas não impactam a performance da aplicação de produção
- Data warehouse pode ter schema diferente (modelo dimensional/star schema) otimizado para analytics



## 2. Performance:

- Data warehouse usa column-store e compressão para queries analíticas rápidas
- Pode ter índices, views materializadas e agregações pré-calculadas

## 3. Flexibilidade:

- Equipa de BI pode fazer queries complexas sem preocupação com impacto
  - Fácil adicionar novas métricas e dimensões
- 

# QUESTÃO 7 (2 valores)

## a) Interpretação da query MongoDB (1.5 valores)

O que a query retorna:

A query retorna os **top 5 hotéis em Portugal com melhor avaliação média** (rating) que tenham **pelo menos 10 reviews desde 1 de janeiro de 2024**, ordenados da melhor para a pior avaliação.

Explicação de cada operador:

### 1. Primeiro \$match:

```
{ country: "Portugal", "reviews.date": { $gte:
ISODate("2024-01-01") } }
```

Filtra apenas documentos de hotéis em Portugal que tenham **pelo menos uma review** com data  $\geq$  2024-01-01. Isto reduz o número de documentos processados.

## 2. \$unwind:

```
{ $unwind: "$reviews" }
```

"Desdobra" o array `reviews`, criando um documento separado para cada review. Por exemplo, um hotel com 3 reviews torna-se 3 documentos (um por review).

## 3. Segundo \$match:

```
{ "reviews.date": { $gte: ISODate("2024-01-01") } }
```

Depois do \$unwind, filtra apenas os reviews individuais que são de 2024 ou posteriores. Isto é necessário porque o primeiro \$match apenas garante que o hotel TEM pelo menos um review de 2024, mas pode ter outros mais antigos.

## 4. \$group:

```
{  
  _id: "$hotel_name",  
  avg_rating: { $avg: "$reviews.rating" },  
  total_reviews: { $sum: 1 }  
}
```

Agrupa os reviews por nome do hotel e calcula:

- Média dos ratings de todos os reviews de 2024
- Contagem total de reviews de 2024

## 5. Terceiro \$match:

```
{ total_reviews: { $gte: 10 } }
```

Filtra apenas hotéis que têm 10 ou mais reviews desde 2024 (garante significância estatística).

#### 6. \$sort:

```
{ avg_rating: -1 }
```

Ordena por rating médio em ordem descendente (melhor primeiro).

#### 7. \$limit:

```
{ $limit: 5 }
```

Retorna apenas os primeiros 5 resultados (top 5 hotéis).

### b) Por que dois operadores \$match? (0.5 valores)

#### Resposta:

Os dois \$match servem propósitos diferentes e ambos são necessários:

#### Primeiro \$match (antes do \$unwind):

- Filtra **documentos inteiros** (hotéis)
- Otimização: reduz o número de documentos processados antes do \$unwind
- Verifica se o hotel tem ALGUM review de 2024 (não verifica todos)

#### Segundo \$match (depois do \$unwind):

- Filtra **reviews individuais**
- Garante que apenas reviews de 2024 são considerados no cálculo da média

- Necessário porque após \$unwind temos reviews individuais que precisam ser filtrados

### **Por que não usar apenas um?**

Se usássemos apenas o primeiro \$match, após o \$unwind teríamos reviews de 2024 E anteriores misturados, e a média incluiria reviews antigos (errado).

Se usássemos apenas o segundo \$match (depois do \$unwind), processaríamos TODOS os hotéis primeiro, incluindo os que não têm nenhum review de 2024, sendo muito ineficiente.

**Conclusão:** O primeiro \$match é uma **otimização de performance** (reduz dados processados) e o segundo é **necessário para correção** (filtra reviews individuais).